Архитектура процессора, память и NUMA

Вводный блок перед CAS, когерентностью кешей, MESI и sync.Mutex

```
код Go → ISA → CPU → кеши → DRAM → NUMA
```

Зачем backend-разработчику лезть ниже Go

На уровне API atomic и Mutex выглядят как маленькие операции.

Под нагрузкой стоимость определяется уже не строками Go-кода, а памятью, кешами и взаимодействием ядер.

Чтобы понять конкуренцию, ложное разделение и цену CAS, нужна аппаратная модель.

```
atomic / Mutex → кеш-линия → когерентность → шина → задержка
```

Иерархия хранения данных



Чем ближе к исполнительным блокам, тем меньше задержка.

Чем ближе уровень — тем меньше доступный объем.

SSD/HDD находятся за пределами иерархии памяти процессора.

Регистры: данные уже внутри ядра

```
mov rax, 10
mov rbx, 20
add rax, rbx

RAX = 30
```

Регистры — очень маленькое и очень быстрое хранилище непосредственно внутри ядра процессора.

Исполнительные блоки работают со значениями в регистрах.

Архитектурные регистры видны ISA; физических регистров внутри процессора с OoO обычно больше.

```
архитектурный RAX → физический регистр 41
следующая версия RAX → физический регистр 73
```

Стек, хип и кеш — разные уровни модели

Логическая организация программы

стек / хип / глобальные данные

Аппаратная иерархия

регистры / L1 / L2 / L3 / DRAM

```
объект из хипа сейчас может быть в L1
локальная переменная может остаться только в регистре
```

Зачем процессору кеш

CPU работает в масштабе долей наносекунды; DRAM — десятков наносекунд.

Если каждое чтение ходит в DRAM, ядро тратит сотни тактов на ожидание.

Кеш хранит копии недавно и часто используемых участков памяти.

```
DRAM: 0x1000 → 42
              ↓
L1:   0x1000 → 42
```

L1, L2, L3 / LLC

L1
десятки KiB
несколько тактов

L2
сотни KiB – MiB
десятки тактов

L3 / LLC
MiB – десятки MiB
десятки–сотни тактов

L1 обычно разделен на L1I и L1D.

Кеш L2 обычно ближе к конкретному ядру.

L3 часто общий для нескольких ядер, но топология бывает сложнее.

Кеш-линия — единица перемещения данных



Процессор кеширует не отдельный int64, а целую кеш-линию.

Типичный размер кеш-линии на x86-64 и многих ARM — 64 байта.

Это приводит нас к ложному разделению кеш-линии.

Локальность: почему кеш работает

Временная локальность

недавно использованные данные часто нужны снова

Пространственная локальность

после адреса X часто нужны данные рядом с X

```
for i := range values {
    sum += values[i]
}
```

Попадание в кеш, промах и отложенная запись

```
Попадание L1
      ↓
при промахе → L2
      ↓
при промахе → L3 / LLC
      ↓
при промахе → DRAM
```

Одно чтение может стоить несколько тактов или несколько сотен тактов.

При отложенной записи процессор обновляет кеш-линию, а DRAM обновить позже.

Поэтому некоторое время кеш содержит новое значение, а DRAM — старое.

```
кеш  = новое значение
DRAM = старое значение
```

DRAM: большая рабочая память

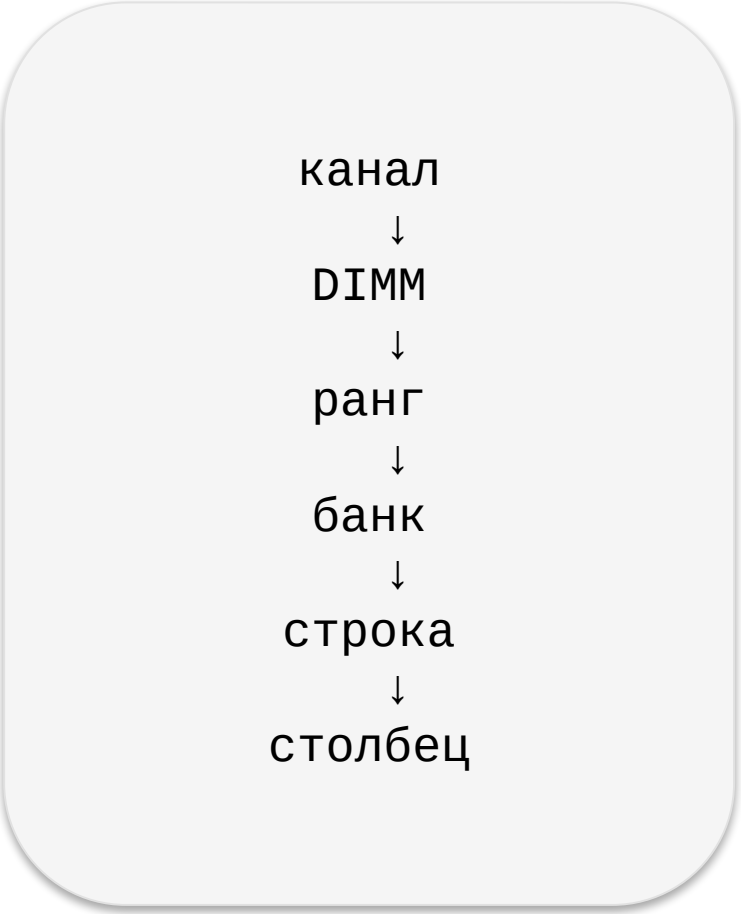
DRAM — динамическая память с произвольным доступом: основная ОЗУ.

Она плотнее и дешевле SRAM, но медленнее кеша процессора.

Ячейки требуют периодического обновления заряда.



У DRAM тоже есть внутренняя топология



Для программы ОЗУ выглядит как адрес → значение.

Физически доступ идет через контроллер памяти и структуру DRAM.

Попадание в открытую строку дешевле ее переключения.

Для этой лекции достаточно понимать: DRAM сама не является плоской и одинаково быстрой.

SSD и HDD: это уже накопители

NVMe SSD
NAND + контроллер
десятки–сотни мкс

SATA SSD
NAND + SATA
сотни мкс

HDD
магнитные пластины + механика
миллисекунды

```
SSD/HDD → ввод-вывод → ядро ОС → DRAM/кеш страниц → кеши CPU → регистры
```

SSD и HDD — постоянное хранилище, а не уровень кеша процессора.

Сеть: следующий временной масштаб

```
процесс → ядро ОС → сокет → NIC → сеть → удаленный узел → приложение
```

Сетевой запрос включает не только передачу байтов, но и работу нескольких программных и аппаратных слоев.

RTT может измеряться от десятков микросекунд в локальной инфраструктуре до десятков и сотен миллисекунд через Интернет.

Это другой масштаб по сравнению с кешем процессора.

Задержка и пропускная способность — разные вещи

Задержка

Сколько ждать одну операцию?

Пропускная способность

Сколько данных передается за единицу времени?

NVMe дает несколько ГБ/с, но случайное чтение занимает десятки мкс.

Сеть 100 Гбит/с не отменяет RTT.

Порядок задержек

Операция	Примерная задержка	При CPU ~4 ГГц
Регистры / простая операция	~0,25–1 нс	~1–4 такта
Попадание в L1	~1 нс	~4 такта
Попадание в L2	~3–5 нс	~12–20 тактов
L3 / LLC	~10–30 нс	~40–120 тактов
DRAM	~60–120 нс	~240–480 тактов
Удаленная DRAM (NUMA)	~100–300+ нс	сотни–1000+ тактов
Случайное чтение NVMe	~50–200 мкс	сотни тысяч тактов
Случайный доступ HDD	~5–15 мс	десятки миллионов тактов
Сетевой RTT	~0,1–200+ мс	миллионы–сотни миллионов тактов

Если L1 = 1 секунда

L1	1 секунда
L2	несколько секунд
L3	десятки секунд
DRAM	1–2 минуты
NVMe	около суток
HDD	несколько месяцев
Сеть	от месяцев до лет

Это масштабная модель порядков задержки, а не таблица измерений.

ISA и микроархитектура

ISA

что процессор гарантирует программе инструкции, регистры, адресация, порядок операций

Микроархитектура

как процессор это реализует конвейер, кеши, OoO, предсказание

`Intel Core и AMD Zen: одна ISA x86-64, но разная микроархитектура`

CISC и RISC

CISC
Сложная система команд

богатые инструкции
пример: x86

RISC
Сокращенная система команд

загрузка / вычисление / запись
ARM, MIPS, PowerPC, RISC-V

Современный RISC-процессор не обязан быть простым.

Современный x86 декодирует многие инструкции в микрооперации.

x86-64 и ARM64

x86 / x86-64

8086 → ... → x86-64
CISC
AMD64 = x86-64
Go: GOARCH=amd64

ARM / AArch64

RISC
модель загрузки/записи
Apple Silicon, Graviton, Ampere
Go: GOARCH=arm64

`Разные ISA → разные инструкции CPU → одна семантика Go`

Другие архитектурные линии

MIPS — классическая RISC-архитектура с моделью загрузки/записи.

PowerPC / POWER — RISC-линия IBM; Apple, GameCube/Wii, Xbox 360, PS3, серверы IBM.

SPARC — RISC от Sun Microsystems, известна регистровыми окнами.

RISC-V — открытая модульная ISA; расширение A добавляет атомики.

DEC Alpha — 64-битная RISC со слабой моделью памяти.

IBM z/Architecture — архитектура мейнфреймов; Go: GOARCH=s390x.

Itanium / IA-64: другой путь

IA-64 ≠ x86-64

Intel и HP: EPIC — явный параллелизм инструкций.

Идея: искать параллелизм инструкций заранее, в компиляторе.

Проблема: компилятор не предскажет промахи кеша, ветвления и данные.

Itanium не вытеснил x86-64.

Чиплеты и 3D-кеш

CCD 0
ядра + кеш

CCD 1
ядра + кеш

3D-кеш
слои SRAM
больше LLC

Кристалл I/O
IMC + PCIe

Чиплеты улучшают выход годных и масштабирование, но топология становится сложнее.

3D-кеш в основном увеличивает емкость LLC; базовая модель L1/L2/L3 не меняется.

UMA и NUMA

UMA

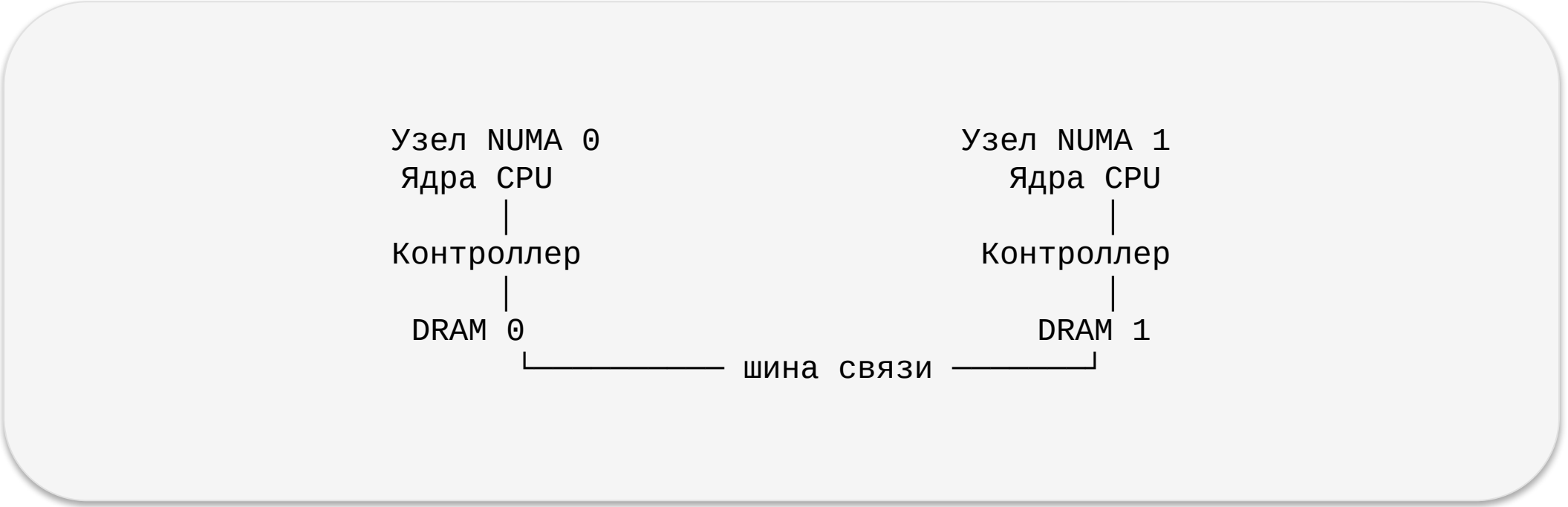
ядра → общий контроллер памяти → ОЗУ
примерно одинаковая стоимость доступа

NUMA

узел 0 ↔ шина ↔ узел 1
локальная память быстрее удаленной

`NUMA = неоднородный доступ к памяти`

NUMA: локальная и удаленная память



Локальная память: доступ через свой контроллер.

Удаленная память: запрос идет через шину к другому контроллеру.

Удаленный доступ медленнее и расходует пропускную способность шины.

AMD Infinity Fabric и Intel UPI

AMD
Infinity Fabric

чиплеты / память / когерентность / I/O обмен

Intel
UPI между сокетами
+ внутренняя шина / сетка

Шина переносит данные.

NUMA — свойство топологии памяти.

Чиплет не равен узлу NUMA: физическая топология может отличаться от видимой ОС.

Возвращаемся к Go

```
код Go
  ↓
компилятор
  ↓
ISA
  ↓
микроархитектура CPU
  ↓
регистры / L1 / L2 / L3
  ↓
DRAM / NUMA
```

```
несколько ядер
  ↓
копии одной кеш-линии
  ↓
когерентность кешей
  ↓
MESI
  ↓
CAS / конкуренция атомиков
  ↓
sync.Mutex
```

Далее: когерентность кешей → MESI → CAS → ложное разделение → sync.Mutex.